Individual Project 2: Streaming Data Pipeline with Pub/Sub Architecture

CS367

Cassandra Mason

10/24/24

Table of Contents

Project Description 3
Pub/Sub Architecture 5
Pseudocode of Pub/Sub Architecture 5
Coding Implementation 8
Results 10
Conclusion 11
References 12

Project Description

This is a design proposal and simple implementation of a streaming data platform for a renewable energy company, REC. This is a fictitious company used to demonstrate the concept and utility of streaming data. The design focuses on publish subscribe (pub/sub) architecture to stream energy production data, weather data, equipment data, and grid data for the company to leverage in decision-making. It highlights the sources of data (producers), the streaming data pipeline, and the places where the data is analyzed and used (subscribers). It will include a description of the system, a visual of the pipeline, the pseudocode design of the pub/sub architecture, the coding implementation of the pub/sub architecture, the results of such a design, and a conclusion.

REC is a renewable energy company that has wind and solar farms across the United States. The wind turbines and solar panels are constantly sending data about their energy output and equipment functions. REC also has small weather monitoring stations at every site that constantly measures temperature, humidity, wind speed, wind direction and other weather conditions that may impact the equipment and/or energy production.

The company needs data from these sources to respond to events in real time and make business decisions about their company. They need to know how much energy they are producing, from which sources, how much energy the grid is demanding, the effect of weather on energy production/consumption, when there are equipment malfunctions, how to maximize efficiency, how to grow their business, etc. As well as the data their proprietary equipment produces, REC would also like to pull in data from weather stations and the grid at-large to help answer some of these questions.

The data sources in this project are the publishers in the pub/sub streaming data architecture. For REC, the data sources are:

- REC wind turbines
- REC solar panels
- REC weather stations
- National weather data streaming database
- National grid data streaming database

The turbines and solar panels produce power output data as well as monitoring data about the status and performance of the equipment to inform maintenance requirements. The weather sensors provide data on the local conditions that may affect the equipment. The weather database gives a broader picture of weather conditions and their impact on both energy production and energy consumption (when paired with the energy data). The grid data reveals information about production and consumption of electricity.

The REC devices are Industrial Internet of Things (IIoT). Their data is packaged into JSON files at the device, then sent over a network, such as cellular, ethernet, or Wi-Fi. It is transferred using a standardized communication protocol such as HTTPS. The data is received by a message broker, such as Apache Kafka, to hold the publisher data for the subscribing applications.

To access the source databases requires an external API that ingests their data into this pipeline. Regular API calls to their databases allow for a near continuous stream of data to match the continuous stream of data coming in from the REC devices.

Data is divided into topics, and subscribers subscribe to specific topics of interest. The topics for this system are:

- Energy-production
- Weather-data
- Equipment-status
- Grid-status

The energy-production topic contains data about power output for solar panels and wind turbines. Overall energy consumption, energy production, the mix of different energy sources, and carbon dioxide emissions.

The data users in this project are the subscribers in the pub/sub streaming data architecture. They subscribe to certain topics and ingest that data to provide insights and store data. The subscribers are:

- Real-time data analytics
- Streaming data dashboard
- Database for storage and historical analysis
- Alerting system

Real time analytics process the data as it arrives to gain insights. Apache Spark is a tool to use to analyze streaming data. REC wants to use real time analytics with machine learning to predict how/when equipment failures or shutdowns may occur and predict the effect of weather on energy production and consumption. Streaming data dashboards will visually show data as it arrives: power production, grid load, rolling averages, temperature, cloud cover, etc. This will help REC and their customers understand their energy landscape. A storage database will subscribe to the message broker to store historical data for historical analysis, reporting, and trend tracking. Such a database could be Amazon S3 or Hadoop Distributed File System. The last subscriber is a system that monitors the equipment status and weather to trigger real-time alerts for maintenance teams when equipment malfunctions, fails, or other anomalies in the data.

Pub/Sub Architecture

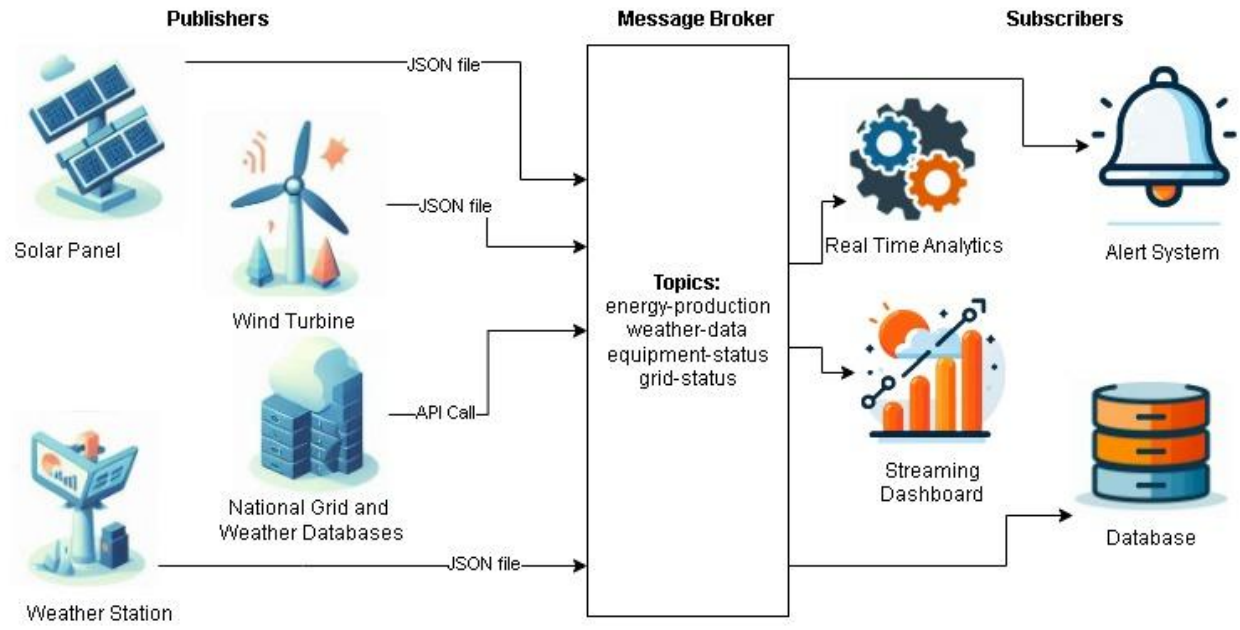


Image generated with draw.io (n.d.).

This flowchart shows the pub/sub architecture of the REC streaming software design. It includes the publishers, message broker with message topics, and the subscribers. The publishers are the solar panels, wind turbines, weather stations, and databases for national weather and grid data. The IIoT devices send information in JSON files, while the database data is accessed with regular API calls. The topics are energy-production, weather-data, equipment-status and grid-status. The subscribers to these topics are real time analytics applications, an alert system, streaming dashboards, and a database to store data.

Pseudocode of Pub/Sub Architecture

Pseudocode to create a publisher class:

```
CLASS Publisher:
        FUNCTION initialize(broker, topic):
                SET broker to broker
                SET topic to topic
        FUNCTION generate_data():
                WHILE True:
                        data = collect_data()
                        publish_to_broker(topic, data)
                        WAIT for 1 second
        FUNCTION publish_to_broker(topic, data):
                SEND data to broker under topic
```

```
            PRINT "Data published to topic:  ", topic
END CLASS

CLASS Subscriber:
    FUNCTION initialize (broker, topic):
        SET broker to broker
        SET topic to topic
    FUNCTION subscribe(topic):
        WHILE True:
            messages = pull_data_from_broker(topic)
            FOR each message IN messages:
                process_message(message)
            WAIT for 1 second
    FUNCTION pull_data_from_broker(topic):
        RETURN GET messages from broker under topic
    FUNCTION process_message(message):
        PRINT "Processing message: ", message
END CLASS
```

This pseudocode sets up the publisher and subscriber classes. The publisher class contains several functions. The "initialize" function creates an instance of a publisher that sets the broker it connects with and the topic(s) it publishes to. The "generate_data" function pulls data from the device or database, assigns it to a topic, and sends it to the message broker using the function "publish_to_broker". The subscriber class contains functions to initialize an instance of a subscriber and subscribe to certain topics. The subscribe function calls a function that pulls the messages from a certain topic from the broker and processes the messages.

Pseudocode to initialize the publishers in the REC streaming software system and begin the streaming process:

```
Solar_panel = Publisher(broker = "message_broker_url", topic = ["energy-production", "equipment-status"])
CALL Solar_panel.generate_data()

Wind_turbine = Publisher(broker = "message_broker_url", topic = ["energy-production", "equipment-status"])
CALL Wind_turbine.generate_data()

Weather_station = Publisher(broker = "message_broker_url", topic = ["weather-data", "equipment-status"])
CALL Weather_station.generate_data()

Grid_database = Publisher(broker = "message_broker_url", topic = ["energy-production", "grid-status"])
```

```
CALL Grid_database.generate_data()

weather_database = Publisher(broker = “message_broker_url”, topic = “weather_data”)
CALL weather_database.generate_data()
```

Each publisher is denoted by name and assigned to the relevant topics for the type of data it produces.

Below is the pseudocode for initializing the subscribers in the system and starting the consuming process.

```
alert_system = Subscriber(broker = “message_broker_url”, topic = [“equipment-status”, “weather-data”]
CALL alert_system.subscribe(“equipment-status”)
CALL alert_system.subscribe(“weather-data”)

streaming_dashboard = Subscriber(broker = “message_broker_url”, topic = [“equipment-status”, “weather-data”, “energy-production”, “grid-status”]
CALL streaming_dashboard.subscribe(“equipment-status”)
CALL streaming_dashboard.subscribe(“weather-data”)
CALL streaming_dashboard.subscribe(“energy-production”)
CALL streaming_dashboard.subscribe(“grid-status”)

analytics = Subscriber(broker = “message_broker_url”, topic = [“equipment-status”, “weather-data”, “energy-production”, “grid-status”]
CALL analytics.subscribe(“equipment-status”)
CALL analytics.subscribe(“weather-data”)
CALL analytics.subscribe(“energy-production”)
CALL analytics.subscribe(“grid-status”)


database_storage = Subscriber(broker = “message_broker_url”, topic = [“equipment-status”, “weather-data”, “energy-production”, “grid-status”]
CALL database_storage.subscribe(“equipment-status”)
CALL database_storage.subscribe(“weather-data”)
CALL database_storage.subscribe(“energy-production”)
CALL database_storage.subscribe(“grid-status”)
```

The above pseudocode initializes all the connections to subscribing applications referencing the message broker and the topics they subscribe to. It then starts the flow of data into these applications by calling the subscribe function for each topic the subscriber follows.

Coding Implementation

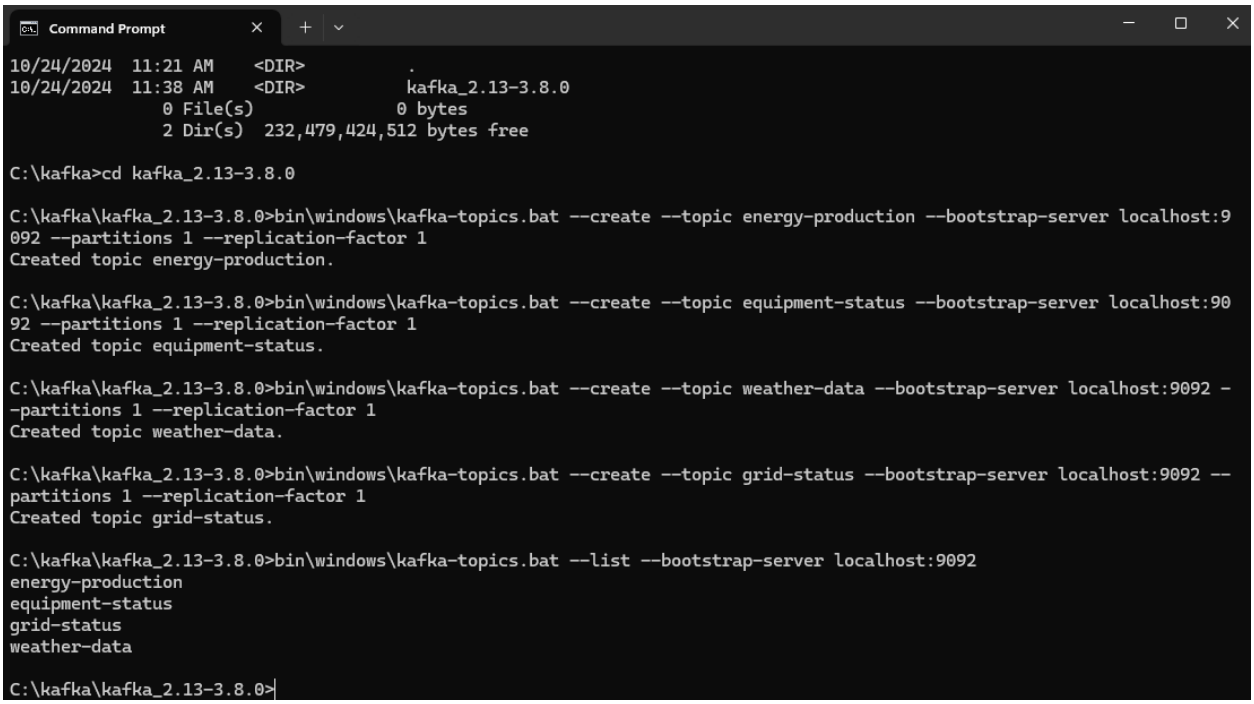
Install Kafka and start zookeeper in a command prompt window:



Start the Kafka Broker in a command prompt window:



Create the Kafka topics for the REC Kafka message broker in a command prompt window:



Once Kafka is up and running, in Spyder I imported the libraries needed to communicate with Kafka and created a variable storing the port number where Kafka is running:

```python
# -*- coding: utf-8 -*-
"""
Created on Wed Oct 16 14:36:24 2024

@author: cassi
"""
# import relevant libraries
import time
from kafka import KafkaProducer
from kafka import KafkaConsumer
import json

# define broker url
message_broker_url = "localhost:9092"
```

Create a class for producers:

```python
# create the publisher class, connecting it to the kafka message broker
class Publisher:
    def __init__(self, broker, topics):
        self.producer = KafkaProducer(
            bootstrap_servers=broker,
            value_serializer=lambda v: json.dumps(v).encode('utf-8')
        )
        self.topics = topics

    def generate_data(self):
        while True:
            data = self.collect_data(topic)
            self.publish_to_broker(topic, data)
            time.sleep(2)

    def collect_data(self, topic):
        """Simulates data collection for each topic"""
        if topic == "energy-production":
            return {"energy_output": 500}  # Example data
        elif topic == "equipment-status":
            return {"status": "operational"}  # Example data
        elif topic == "weather-data":
            return {"temperature": 24.5, "wind_speed": 12.3}  # Example data
        elif topic == "grid-status":
            return {"grid_load": 75}  # Example data
        else:
            return {"default_data": "value"}  # Default data

    def publish_to_broker(self, topic, data):
        # Send data to the broker
        self.producer.send(topic, data)
        print(f"Data published to topic '{topic}': {data}")
```

Create a class for subscribers:

```python
# create the subscriber class, connecting it to the kafka message broker
class Subscriber:
    def __init__(self, broker, topic):
        self.broker = broker
        self.topic = topic

    def subscribe(self):
        while True:
            messages = self.pull_data_from_broker(self.topic)
            for message in messages:
                self.process_message(message)
            time.sleep(1)

    def pull_data_from_broker(self, topic):
        # Simulate pulling data from the broker
        return ["message1", "message2"]

    def process_message(self, message):
        print(f"Processing message: {message}")
```

Instantiate the different producers as their own objects:

```python
# Instantiate the publishers for the renewable energy streaming data system
solar_panel = Publisher(broker=message_broker_url, topics=["energy-production", "equipment-status"])
wind_turbine = Publisher(broker=message_broker_url, topics=["energy-production", "equipment-status"])
weather_station = Publisher(broker=message_broker_url, topics=["weather-data", "equipment-status"])
grid_database = Publisher(broker=message_broker_url, topics=["energy-production", "grid-status"])
weather_database = Publisher(broker=message_broker_url, topics=["weather-data"])

solar_panel.generate_data()
```

Start an example data stream from the solar panel object:

```python
solar_panel.generate_data()
```

(*Apache Kafka*, n.d.; *Kafka-Python*, n.d.; Contributor, 2024; Simplilearn, 2017)

Results

This project was implemented using Kafka as the message broker between the publishers and subscribers. A publish and subscribe class were created to make the relevant instances of publishers and subscribers to the REC system.

I ran out of time to make the code work, I need to define a variable 'topic' as well as topics, so when I ran the code I have, I got this error:

```
In [19]: runfile('C:/Users/cassi/OneDrive/Documents/School/CTU/Streaming_data_analytics/Unit2/IP2_pub_nub.py', wdir='C:/Users/cassi/OneDrive/Documents/School/CTU/Streaming_data_analytics/Unit2')
Traceback (most recent call last):

  File "C:\Users\cassi\OneDrive\Documents\School\CTU\Streaming_data_analytics\Unit2\IP2_pub_nub.py", line 76, in <module>
    solar_panel.generate_data()

  File "C:\Users\cassi\OneDrive\Documents\School\CTU\Streaming_data_analytics\Unit2\IP2_pub_nub.py", line 27, in generate_data
    data = self.collect_data(topic)

NameError: name 'topic' is not defined
```

Currently a simple data stream model is used to replicate data. Future work would include using APIs for streaming databases to pull in real-world, relevant data streams. The APIs of interest are OpenWeather's Weather API (*Weather API - OpenWeatherMap*, n.d.), NREL's renewable energy

database (*NREL: Developer Network*, n.d.), and Electricity Maps for grid data (*Get Our Data | Electricity Maps*, n.d.).

Future work also includes creating a scatterplot visual of incoming energy production, temperature, and energy consumption data using Apache Spark. This would show a rolling window of streaming data starting from the most recent data to a day back. It would be part of the streaming dashboard subscriber application, and benefit users by showing them in real time what data is being pulled into the system, and what values they have.

I also want to run the different producers on separate threads so they can all run simultaneously.

Conclusion

This pub/sub streaming data software design for the renewable energy company, REC, allows for real-time data to inform decisions about energy production, grid utilization, and the effect of weather on supply and demand. The publishers that provide streaming data include renewable energy devices: solar panels and wind turbines, on-site weather stations, and streaming databases about overall weather and grid usage.

They send data to different topics. The energy production topic can tell us how much power is generated by solar panels, wind turbines, the total power output to the grid, etc. The equipment status topic lets maintenance teams at REC monitor the equipment for failures, malfunctions, wearing, and unsafe weather conditions for equipment operation that lets them take care of the equipment better. The weather data topic allows for prediction models to determine the effect of weather conditions on power production and consumption, and what weather variables are unsafe for equipment operation. The grid-status topic gives information about overall supply and demand, when and where power is most needed, where there are surpluses, what the overall makeup of power generation is (renewable vs. nonrenewable) and long-term strategies for grid optimization.

Subscribers to these topics process the data streams to create visual dashboards about what is happening with REC's equipment and power generation, grid production and consumption, and weather trends. An alerting system subscriber monitors the equipment status data and automatically sends alerts or shuts down devices when there is a red flag. Real-time analytics looks for trends and makes predictions from the incoming data streams. The processed data is stored in a database for historical analysis, record keeping, and reports.

References

Apache Kafka. (n.d.). Apache Kafka. Retrieved October 24, 2024, from https://kafka.apache.org/downloads

Contributor, G. (2024, January 29). *How to install and configure Apache Kafka on Windows*. Sling Academy. https://www.slingacademy.com/article/how-to-install-and-configure-apache-kafka-on-windows/

Draw.io. (n.d.). Retrieved August 23, 2024, from https://www.drawio.com/

Get our data | Electricity Maps. (n.d.). Retrieved October 24, 2024, from https://www.electricitymaps.com/get-our-data

kafka-python: Pure Python client for Apache Kafka (Version 2.0.2). (n.d.). [Python]. Retrieved October 24, 2024, from https://github.com/dpkp/kafka-python

NREL: Developer Network. (n.d.). Retrieved October 24, 2024, from https://developer.nrel.gov/

Simplilearn (Director). (2017, July 25). *Kafka Tutorial | Apache Kafka Tutorial For Beginners | Kafka Architecture |What Is Kafka|Simplilearn* [Video recording]. https://www.youtube.com/watch?v=U4y2R3v9tlY

Weather API - OpenWeatherMap. (n.d.). Retrieved October 24, 2024, from https://openweathermap.org/api